CENG218 – Lab Project 2

Document Indexing

Due: 17/05/2026 – 23:59

In this project, you will design a very simple yet effective document retrieval system that will respond to single word queries. This system will allow users to enter a single word, then the system will return a list of documents containing this word.

Consider a scenario where there are 10,000 text files, and you would like to develop a program that will let users search for a particular keyword among all documents. Users are interested in finding documents containing this keyword.

A naive approach to solving this problem would be to scan all files for the keyword and list the documents. However, this is inefficient. A more efficient approach is achieved through indexing, which involves organizing and storing documents for later retrieval based on words they contain.

Design and implement a system that will index the documents by their content words, considering the linked list as a data structure. The system will be menu-driven and will first start indexing by reading file location. Once the index is created, the system will display the time spent in creating the index and then show the following menu.

Menu Options

Simple Document Retrieval System

(Index Creation Time: 30 Minutes)

1. **Enter a Single Keyword to List The Document(s) (File Names)**

2. **Display the Top N Words that Appeared Most Frequently**

3. **Display the Top N Words that Appeared Least Frequently**

4. **Display Word Frequency With Line Numbers**

5. **Display Operation History**

6. **Exit**

Figure 1 and Figure 2 summarizes the aim of this project.

File Name (or Document)	Content of File
1.txt	Pease porridge hot, pease porridge cold.
2.txt	Pease porridge in the pot.
3.txt	Nine days old.
4.txt	Some like it hot, some like it cold.
5.txt	Some like it in the pot.
6.txt	Nine days old.

Figure 1: Small set of files and their content.

File Name (or Document)	Content of File
1.txt	Pease porridge hot, pease porridge cold.
2.txt	Pease porridge in the pot.
3.txt	Nine days old.
4.txt	Some like it hot, some like it cold.
5.txt	Some like it in the pot.
6.txt	Nine days old.

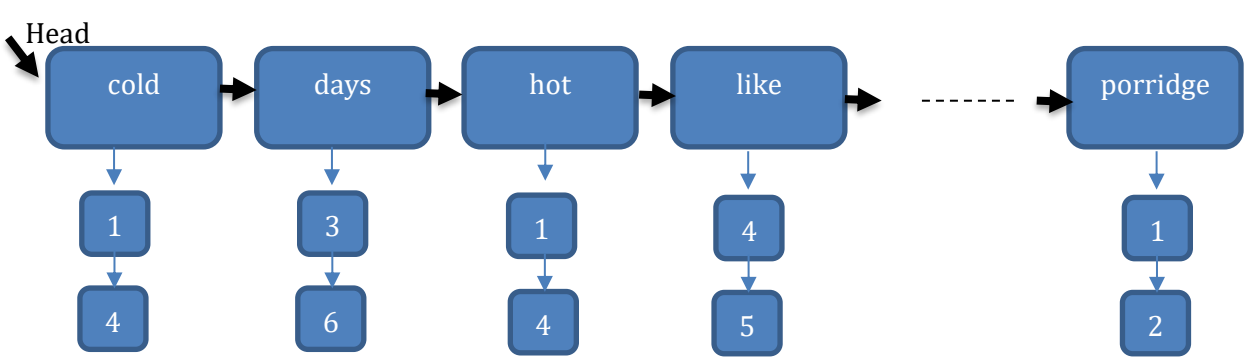
Indexing System

Term	File Name(s)	Line Numbers	Frequency
cold	1; 4	1; 1	1; 1
days	3; 6	1; 1	1; 1
hot	1; 4	1; 1	1; 1
in	2; 5	1; 1	1; 1
it	4; 5	1; 1	2; 1
like	4; 5	1; 1	2; 1
nine	3; 6	1; 1	1; 1
old	3; 6	1; 1	1; 1
pease	1; 2	1; 1	2; 1
porridge	1; 2	1; 1	2; 1
pot	2; 5	1; 1	1; 1
some	4; 5	1; 1	2; 1
the	2; 5	1; 1	1; 1

Explanations:
- **The word "cold":**
- **File 1 (1)**: Line 1, Frequency 1 (Appears 1 time in Line 1)
- **File 4 (4)**: Line 1, Frequency 1 (Appears 1 time in Line 1)
- **The word "porridge":**
- **File 1 (1)**: Line 1, Frequency 2 (Appears 2 times in Line 1)
- **File 2 (2)**: Line 1, Frequency 1 (Appears 1 time in Line 1)

Figure 2: Text Files (Documents) are Indexed by their word contents.

Since we may not easily estimate the number of words among all documents, one possible solution would be to use linked lists to maintain a list of words. Additionally, for each word, a linked list should be used to store the associated file names, line numbers, and word frequencies. This way, each word will have its own linked list of file names, along with the specific line numbers where the word appears and its frequency in each document. The number of documents (or files) may not be easily estimated, and using linked lists allows efficient handling and retrieval of this data as the collection of documents grows or changes.



1. Design Guidelines

1.1. While reading text files (or documents):

i. Lowercase all words

ii. Get all words, where a word is a string of alpha characters terminated by a non-alpha character (white space is not alpha). The alpha characters are defined to be [a-z].

- Therefore, the sequence of characters for the word "apple+78&'^+orange" would be 'apple' and 'orange'

1.2. Limitations and Assumptions:

1. The collection of documents is closed (content and number of documents are fixed and will never change).

2. Each document stored in a single text file. Hence, if there are 10,000 documents, then there are 10,000 text files (Collection of documents is provided on WebOnline as AllDocs.rar).

2. Task Implementation Signatures

These signatures are not mandatory and have been added for illustrative purposes.

- **Document Indexing**

```
// Function to initialize document indexing
void createIndex(const std::string& directory);

// Function to load all documents into memory
void loadDocuments(const std::string& directory);

// Function to add words to the index (linked list)
void addWordToIndex(const std::string& word, const std::string& document, int lineNum);
```

2.1. Searching for a Keyword

```
// Function to search for a keyword in indexed documents
void searchKeyword(const std::string& keyword);

// Function to list documents containing the keyword
void listDocumentsContainingKeyword(const std::string& keyword);
```

2.2. Displaying Top N Most Frequent Words

```
// Function to display top N most frequent words
void displayTopNMostFrequentWords(int N);
```

2.3. Displaying Top N Least Frequent Words

```
// Function to display top N least frequent words
void displayTopNLeastFrequentWords(int N);
```

2.4. Displaying Word Frequency with Line Numbers

```
// Function to display word frequency with line numbers
void displayWordFrequencyWithLineNumbers(const std::string& word);

// Helper function to display word frequency for each document
void displayWordFrequencyInDocument(const std::string& word, const std::string&
document);
```

2.5. Displaying Operation History

```
// Function to display operation history
void displayOperationHistory();

// Function to log operations with timestamp
void logOperation(const std::string& action, const std::string& details);
```

3. Illustrative Task Scenario

3.1. Enter a single keyword to list the document(s)

Search for the documents containing the entered keyword.

This option will return a list of document names where the entered keyword appears.

Example:

If the user searches for the word “pot”, the system would display the documents containing this word, such as 2.txt and 5.txt.

```
Search Results:
Keyword: "pot"
Documents:
- 2.txt
- 5.txt
```

3.2. Display Top N Most Frequent Words

Show the top N most frequent words in the entire document set along with a progress bar indicating the frequency distribution.

This will be done by displaying each word along with a progress bar, where the length of the bar corresponds to the word's frequency relative to a fixed maximum value (e.g., 1500). The **full block character** (■) will be used for the progress bar to visually represent the frequency of each word.

Example:

```
1. banana   [■■■■■■■■■■] (1332/1500)
2. apple    [■■■■■■■■■ ] (1012/1500)
3. grape    [■■■■■■■■  ] (904/1500)
4. peach    [■■■■■■    ] (610/1500)
5. kiwi     [■■■■■     ] (303/1500)
```

• The full block character (■) is used to create a visual bar that indicates the relative frequency of each word.

• The frequency ratio is displayed next to the word in the format:

- *(actual frequency/max frequency)*

This approach helps users to easily compare the relative frequency of different words with the help of both the frequency ratio and the progress bar visualization.

3.3. Display Top N Least Frequent Words

Show the top N least frequent words in the entire document set along with a progress bar indicating the frequency distribution.

Example:

```
1. fig      [■         ] (5/1500)
2. date     [■■        ] (11/1500)
3. lemon    [■■■       ] (24/1500)
4. plum     [■■■■      ] (35/1500)
5. pear     [■■■■■     ] (48/1500)
```

• Fig is the least frequent word, appearing 5 times.

• Pear is the most frequent among the least frequent words, appearing 48 times.

• The full block character (■) is used to visually represent the frequency of each word.

• The frequency is displayed next to the word in the format (actual frequency / maximum frequency).

This allows users to easily observe the least frequent words and understand their relative occurrence across the entire set of documents.

3.4. Display Word Frequency with Line Numbers

Show the frequency of the word in each document and list the specific line numbers where the word appears.

This option should use a linked list to maintain the word frequency and associated line numbers for each document.

Example:

If the user searches for the word "pot", the system would display something like:

```
Word: "pot"
Document: 2.txt
- Line 1: "Pease porridge in the pot." (1 time)
Document: 5.txt
- Line 1: "Some like it in the pot." (1 time)
```

3.5. Displaying Operation History

Track and display the history of operations performed by the user.

The system will log every action performed, including the search queries, word frequencies, and documents involved. Every time an operation is performed (e.g., searching for a word, displaying frequent words), it will be logged with a timestamp and the action details.

Example:

The operation history will include entries like:

```
Operation History:
1. Search Query: "hot"
   - Time: 12:30 PM
   - Action: Displayed frequency of the word "hot" in documents.

2. Search Query: "cold"
   - Time: 12:35 PM
   - Action: Displayed frequency of the word "cold" with line numbers in the documents.

3. Action: Displayed top 5 most frequent words with progress bars.
   - Time: 12:40 PM
```